

Datenstruktur `Queue` in Java | `LinkedList`

Kriterium	Beschreibung	Wann zu verwenden?
FIFO-Prinzip (First In, First Out)	Eine Queue folgt dem First In, First Out (FIFO)-Prinzip, bei dem das zuerst eingefügte Element als erstes entfernt wird.	Wenn eine Warteschlange benötigt wird, bei der die Reihenfolge der Elemente wichtig ist, z.B. bei der Verwaltung von Aufgaben oder Ereignissen.
Dynamische Größe	Eine <code>LinkedList</code> wächst dynamisch, d.h. es muss keine feste Größe definiert werden.	Wenn die Anzahl der Elemente in der Warteschlange zur Laufzeit variiert und eine dynamische Anpassung erforderlich ist.
Effizientes Hinzufügen und Entfernen	Einfügen (am Ende der Queue) und Entfernen (vom Anfang der Queue) erfolgen in $O(1)$ -Zeit, da die <code>LinkedList</code> eine doppelt verkettete Struktur verwendet.	Wenn häufige Einfügungen und Entfernungen von Elementen am Anfang und Ende der Queue erforderlich sind und hohe Effizienz erwartet wird.
Reihenfolge der Elemente	Die Reihenfolge der Elemente bleibt erhalten, wobei das erste eingefügte Element auch als erstes entfernt wird.	Wenn die Reihenfolge von Eingaben und Ausgaben von Bedeutung ist, z.B. bei der Verarbeitung von Aufgaben oder Anfragen.
Speicherverwaltung	Eine <code>LinkedList</code> verwendet dynamische Speicherverwaltung und benötigt für jedes Element zusätzlichen Speicher für Verweise auf die benachbarten Elemente.	Wenn dynamische Speicherverwaltung gewünscht ist und keine feste Anzahl von Elementen bekannt ist.
Kein direkter Zugriff auf Elemente	Eine <code>LinkedList</code> unterstützt keinen direkten Zugriff auf Elemente durch Indizes. Der Zugriff erfolgt nur über Traversieren der Liste.	Wenn nur das Hinzufügen und Entfernen von Elementen benötigt wird und kein direkter Zugriff auf beliebige Elemente erforderlich ist.

Kriterium	Beschreibung	Wann zu verwenden?
Performance bei großem Volumen	Im Vergleich zu einer Array-basierten Queue hat eine <code>LinkedList</code> eine höhere Speichernutzung, da für jedes Element zusätzliche Verweise gespeichert werden müssen.	Wenn die Queue viele Elemente enthält und dynamische Hinzufügungen und Entfernungen erforderlich sind, aber Speicher nicht kritisch ist.
Verwendung in Multithreading	Eine <code>LinkedList</code> ist nicht threadsicher. Für einen sicheren parallelen Zugriff sollte eine synchronisierte Variante oder explizite Synchronisation verwendet werden.	Wenn keine parallelen Zugriffe auf die Queue erfolgen oder Synchronisation manuell gehandhabt werden kann.
Effizienz bei variierenden Datenmengen	Eine <code>LinkedList</code> ist besonders geeignet, wenn die Queue eine stark schwankende Anzahl von Elementen enthalten kann. Die Größe passt sich dynamisch an.	Wenn die Anzahl der Elemente in der Queue sehr variabel ist und eine dynamische Anpassung erforderlich ist.
Verwaltung von <code>null</code>-Werten	Eine <code>LinkedList</code> erlaubt es, <code>null</code> -Werte als Elemente zu speichern.	Wenn <code>null</code> -Werte als gültige Daten in der Warteschlange verarbeitet werden müssen.
Leistungsbeeinträchtigung bei vielen Operationen	Bei vielen gleichzeitig durchgeführten Einfügungen und Löschungen bleibt die Leistung dank der <code>LinkedList</code> mit ihrer $O(1)$ -Komplexität für die häufigsten Operationen stabil.	Wenn eine hohe Anzahl an Operationen in der Warteschlange (Einfügungen und Entfernungen) zu erwarten ist.
Verwendung für Warteschlangen mit variablen Prioritäten	Eine <code>LinkedList</code> eignet sich gut für einfache FIFO-Queues, jedoch ist sie weniger geeignet für Warteschlangen mit komplexeren Prioritätsanforderungen (wie <code>PriorityQueue</code>).	Wenn nur eine einfache FIFO-Verarbeitung der Elemente benötigt wird.

Zusammenfassung

Die Verwendung einer Queue mit einer `LinkedList` ist besonders sinnvoll, wenn:

- Das FIFO-Prinzip (First In, First Out) angewendet werden muss.
- Die Anzahl der Elemente in der Warteschlange variabel ist und dynamisch angepasst werden muss.
- Effizientes Einfügen und Entfernen von Elementen an den Enden der Warteschlange benötigt wird.
- Kein direkter Zugriff auf die einzelnen Elemente erforderlich ist.

